

Hopper (H100/H200) -> NVIDIA B300 (Blackwell Ultra) 迁移摸底报告

OpenAI

2026-09-01

目录

1 执行摘要	3
1.1 最重要的十条结论	3
1.2 建议的首批工程结论	3
2 1. 范围、术语和证据口径	4
2.1 1.1 本报告默认的迁移对象	4
2.2 1.2 三种性能数据不可混用	4
3 2. B300 硬件性能与系统形态	4
3.1 2.1 Blackwell Ultra 芯片级信息	4
3.2 2.2 DGX/HGX B300 节点规格	4
3.3 2.3 H200 -> B300 的“理论上限”变化	5
4 3. NVLink/NVSwitch 拓扑	5
4.1 3.1 HGX B300 Baseboard	5
4.2 3.2 与 HGX H100 的物理差异	5
4.3 3.3 对通信迁移的实际影响	6
5 4. 从 WGMMA 到 TCGen05/TMEM	6
5.1 4.1 为什么不能把 Hopper kernel 原样带过去	6
5.2 4.2 指令吞吐应如何理解	7
5.3 4.3 迁移审计关键词	7
6 5. 1-SM 与 2-SM Cooperative Kernel	7
6.1 5.1 概念	7
6.2 5.2 NVFP4/MXFP4 常见合法 tile	7
6.3 5.3 何时优先 2-SM	8
6.4 5.4 推荐的选择机制	8
7 6. Gather4: 是什么、能做什么、不能做什么	8
7.1 6.1 指令语义	8
7.2 6.2 可探索的工作负载（工程推断）	8
7.3 6.3 什么时候不要用	9
7.4 6.4 最小验证矩阵	9
8 7. NVFP4、MXFP4、MXFP8 与 Block Scaling	9
8.1 7.1 Block scaling 的计算形式	9
8.2 7.2 格式对比	9
8.3 7.3 MXFP8	10
8.4 7.4 MXFP4	10
8.5 7.5 NVFP4	10
8.6 7.6 量化迁移推荐顺序	10

9	8. B300 软件技术栈	11
9.1	8.1 分层技术栈	11
9.2	8.2 建议的版本策略	11
9.3	8.3 CUDA 13.x 已知正确性风险	11
10	9. 推荐文档、教程、论文与开源项目	12
10.1	9.1 第一优先级：官方架构与 ISA	12
10.2	9.2 第二优先级：CUTLASS/CuTe 与可信教程	12
10.3	9.3 低精度与训练/推理 recipe	12
10.4	9.4 Tile/DSL 与迁移生产力	12
10.5	9.5 Runtime、通信与可观测性	13
11	10. NVIDIA Skills：推荐安装与使用方式	13
11.1	10.1 Skills 的定位	13
11.2	10.2 对 H->B 迁移最相关的 Skills	13
11.3	10.3 使用规则	14
12	11. 公开性能数据	14
12.1	11.1 B300 理论/产品规格	14
12.2	11.2 Transformer Engine GEMM 公测数据	14
12.3	11.3 CUDA 软件版本带来的同硬件增益	14
12.4	11.4 MLPerf Inference 6.0：GB300 参考	14
12.5	11.5 MLPerf Training 6.0：HGX B300 实测	15
12.6	11.6 B200 vs H200 的公开下限参考	15
12.7	11.7 公开数据使用规则	15
13	12. 迁移执行计划	15
13.1	Phase 0：冻结 Hopper 基线	15
13.2	Phase 1：B300 平台 bring-up	16
13.3	Phase 2：正确性优先的 kernel 迁移	16
13.4	Phase 3：精度与 1-SM/2-SM 调优	16
13.5	Phase 4：模型与分布式验证	16
13.6	Phase 5：生产可靠性	16
14	13. Benchmark 设计	16
14.1	13.1 Microbenchmark 层	16
14.2	13.2 End-to-end 层	17
14.3	13.3 Nsight 观察项	17
15	14. 重点风险清单	17
16	15. 第一阶段十个工作日建议	18
16.1	Day 1-2：资产与平台	18
16.2	Day 3-4：编译与正确性	18
16.3	Day 5-6：GEMM 与精度	18
16.4	Day 7-8：模型	18
16.5	Day 9：多节点与可靠性	19
16.6	Day 10：Go/No-Go	19
17	16. 最终建议	19
18	附录 A：环境清单模板	19
19	附录 B：资料入口总表	20
19.1	Hardware / system	20
19.2	CUDA / ISA / kernels	20
19.3	Colfax tutorials	20
19.4	Precision / papers	20
19.5	CUDA Tile / DSL / Skills	21

报告定位：面向 H100/H200 (Hopper, sm_90a) 到 B300 (Blackwell Ultra, Compute Capability 10.3, sm_103/sm_103a) 的工程迁移摸底。

资料截止时间：2026-09-01。

证据分级：A = NVIDIA/MLCommons/OCP 官方；B = NVIDIA 官方开源仓库或可信工程教程；C = 工程推断，必须在目标机器实测。

1 执行摘要

1.1 最重要的十条结论

1. **B300 不是“更快的 B200”，更不是只改编译架构号即可。** B300 对应 Compute Capability 10.3, PTX 目标为 sm_103/sm_103a; B200 的核心资料仍然有用，但所有架构特化路径都要单独验证 SM103 支持。见 [PTX ISA](#) 与 [CUTLASS Overview](#)。
2. **硬件上最确定的增量是容量、带宽、FP4 与 NVLink。** Blackwell Ultra 芯片上限为 160 SM、640 个第五代 Tensor Core、288 GB HBM3e、8 TB/s HBM，以及 1.8 TB/s 双向 NVLink；但实际可用 SM、HBM、频率和功耗随 SKU 而变化。见 [Blackwell Ultra 架构博客](#)。
3. **HGX/DGX B300 节点内是 8 GPU + 2 个第五代 NVSwitch。** 每个 GPU 的 18 条 NVLink 分成两组，各 9 条连接两个 NVSwitch；节点总注入带宽口径为 14.4 TB/s。H100 HGX Baseboard 则是 4 个 NVSwitch，18 条链路约按 4/5/5/4 分散。见 [Fabric Manager 拓扑表](#)。
4. **Hopper 的 wgmma.mma_async 不能视为可直接平移到 Blackwell。** Blackwell 核心路径变为 tcgen05.mma (CUTLASS 常称 UMMA)，累加器进入专用 Tensor Memory (TMEM)，由单线程发起异步 MMA，随后通过 tcgen05.ld 将结果搬回寄存器做 epilogue。见 [PTX ISA](#) 与 [Colfax TMEM 教程](#)。
5. **1-SM 应作为 bring-up 基线，2-SM 应作为按 shape 选择的优化。** 2-SM/CTA-pair 能提升数据复用和算术强度，但增加成对调度、cluster、barrier、multicast 和边界处理复杂度；小 M、skinny GEMM、ragged/grouped GEMM、decode 长尾不一定获益。
6. **NVFP4、MXFP4、MXFP8 不是“同一种低精度的不同名字”。** 它们的元素格式、scale 类型、block 粒度、布局 and 训练/推理 recipe 不同；checkpoint 不能只改 dtype 名称，通常需要重新量化、重新打包和重新校准。
7. **优先迁移顺序建议为 BF16 -> FP8 parity -> MXFP8 -> NVFP4，最后再评估 MXFP4。** 先把正确性、框架和通信跑稳，再逐级引入 block scaling，能明显降低定位难度。
8. **INT8 是需要单独立项的红旗。** PTX 9.3 的 tcgen05.mma Target ISA Notes 没有把 .kind::i8 列入 sm_103a；这不表示 B300 上所有 INT8 工作负载都不能运行，但表示不能默认存在与 H100 自定义 WGMMMA INT8 路径——对应的 TCGen05 原生路径，必须检查库选路和回退。
9. **CUDA/cuBLAS 版本影响正确性，不只是性能。** CUDA 13.2 Update 1 曾有 NVFP4 tensor-wide scaling 被忽略的严重 cuBLAS 问题；后续版本还修复了 B300/GB300 上 M 非 64 倍数时 NVFP4 结果错误。迁移必须固定已修复的容器/驱动，并加入 shape sweep 与并发 TMEM 回归。见 [CUDA Release Notes](#)。
10. **公开峰值只能用于容量规划，公开 benchmark 只能用于方向判断。** 真正的迁移结论必须来自你们自己的 shape histogram、prefill/decode 分离、MoE 专家负载分布、通信尺寸分布、质量指标和功耗数据。

1.2 建议的首批工程结论

项目	默认决策	原因
架构目标	独立维护 sm_90a 与 sm_103a 路径	避免架构特化指令和布局被错误复用
Kernel bring-up	库/CUTLASS 路径优先，1-SM 优先	缩小正确性风险与调试面
2-SM	只对真实高频 shape 开启 autotune	对小/不规则 shape 可能负收益
低精度	BF16/FP8 验证后再上 MXFP8/NVFP4	将硬件、软件和数值误差分层
CUDA 版本	使用当前 B300 认证、已修复的 CUDA 13.x NGC 镜像并锁 digest	避免已知 cuBLAS 正确性问题
通信	先跑 NCCL P2P + collectives，再跑模型	先隔离 NVLink/NVSwitch/NIC/NUMA 问题
公开性能	只做预期区间，不作验收值	软件版本、精度、shape、场景差异很大

2 1. 范围、术语和证据口径

2.1 1.1 本报告默认的迁移对象

- 源端：H100 SXM/H200 SXM, Hopper, Compute Capability 9.0, 常见架构特化目标 sm_90a。
- 目标端：B300 SXM, Blackwell Ultra, Compute Capability 10.3, 目标 sm_103/sm_103a。
- 系统形态：重点讨论 8-GPU HGX/DGX B300; GB300 NVL72 只作为 rack-scale 性能与软件成熟度的参考。
- 工作负载：大模型训练、推理、MoE、Attention、GEMM、量化与多 GPU 通信。

2.2 1.2 三种性能数据不可混用

- 芯片理论峰值**：例如 15 PFLOPS dense NVFP4/GPU, 适合判断上限, 不代表应用可达到。
- 系统产品规格**：例如 DGX B300 108 PFLOPS dense FP4、14.4 TB/s aggregate NVLink, 包含产品 SKU 和功耗约束。
- 实测结果**：例如 MLPerf、Transformer Engine GEMM benchmark, 必须同时看精度、模型、shape、并行策略、运行时和量化开销。

口径提醒：NVIDIA 当前公开页面对 DGX B300 HBM 有 “8 x 288 GB = 2.3 TB” 和 “Total GPU Memory 2.1 TB” 两种表述；芯片博客给出的是 max capacity, 产品页更接近当前系统可用/销售口径。不要自行把差异归因于某一个固定原因；采购与容量设计以实际 OEM BOM、nvidia-smi -q 和容器内可分配显存为准。

3 2. B300 硬件性能与系统形态

3.1 2.1 Blackwell Ultra 芯片级信息

指标	B300 / Blackwell Ultra 公布值	迁移意义
Compute Capability	10.3, sm_103/sm_103a	必须单独编译与测试架构特化代码
最大 SM 数	160 (随 SKU 变化)	launch/grid/occupancy 模型需重新测
第五代 Tensor Core	640	tcgen05/TMEM 是主要新路径
双 die 互联	NV-HBI 10 TB/s	对 CUDA 呈现为一个 device, 但存在片上结构差异
HBM3e	最大 288 GB/GPU	约为 H200 141 GB 的 2.04 倍
HBM 带宽	最大 8 TB/s/GPU	约为 H200 4.8 TB/s 的 1.67 倍
NVLink 5	1.8 TB/s 双向/GPU	为 Hopper NVLink 4 的 2 倍
PCIe	Gen6 x16, 256 GB/s 双向	Host/NIC/GPU 路径上限提高
NVFP4	15 PF dense / 20 PF sparse (芯片上限)	推理和低精度训练的主要增量
FP8	5 PF dense / 10 PF sparse (芯片上限)	FP8 峰值相对 B200并未同幅提升
Attention SFU EX2	10.7 TeraExponentials/s	softmax/exp 相关路径有专用增量
最大 TGP	up to 1400 W	机房供电、散热、功耗封顶不可忽略

主要来源：[Blackwell Ultra 技术博客](#)、[PTX ISA](#)、[CUTLASS Overview](#)。

3.2 2.2 DGX/HGX B300 节点规格

指标	DGX/HGX B300	工程含义
GPU	8 x B300 SXM	单节点常见并行域仍为 8 GPU
GPU memory	官方页面存在 2.1 TB 与 2.3 TB 两种口径	以实机可用容量为准
FP4 system peak	144 PF sparse / 108 PF dense	不要把 144 PF 当成 dense

指标	DGX/HGX B300	工程含义
FP8 system peak NVSwitch	72 PF sparse; dense 为其一半 2 x fifth-gen NVSwitch	比较时必须注明 sparsity 与 H100 的 4-switch 物理结构不同
Aggregate NVLink	14.4 TB/s	8 x 1.8 TB/s 注入带宽口径，不是单 GPU pair 带宽
Cluster networking Storage/management	8 x 800 Gb/s ConnectX-8 2 x 400 Gb/s BlueField-3	GPUDirect/NIC affinity 更重要 存储、管理和数据面路径需分离验证
OS	DGX OS 7 / Ubuntu 24.04 LTS; 亦支持 RHEL/Rocky	驱动、Fabric Manager 与容器矩阵要锁定
Power	约 14-14.5 kW	单节点功耗约为传统 8xH100 节点显著提升
Form factor	10U	上架、PDU、风冷/液冷与机柜密度要提前确认

来源：DGX B300 User Guide、DGX B300 产品页、HGX Platform。

3.3 2.3 H200 -> B300 的“理论上限”变化

维度	H200	B300 最大/公布值	约数
HBM 容量	141 GB	288 GB	2.04x
HBM 带宽	4.8 TB/s	8 TB/s	1.67x
NVLink 双向/GPU	0.9 TB/s	1.8 TB/s	2.0x
最大 TGP	up to 700 W	up to 1400 W	2.0x

这张表只说明硬件资源变化，不说明端到端速度。decode、MoE、小 GEMM、通信长尾和 CPU 调度往往无法按峰值线性缩放。

4 3. NVLink/NVSwitch 拓扑

4.1 3.1 HGX B300 Baseboard

NVIDIA Fabric Manager 文档明确给出：**HGX B200/B300 上，每个 GPU 有 9 条 NVLink 连接每一个 NVSwitch。**因此每个 GPU 合计 18 条链路。

HGX/DGX B300: 8 GPU + 2 NVSwitch

```

GPU0  -- 9 links --> NVSwitch 0 <-- 9 links -- GPU1 ... GPU7
|
+--- 9 links --> NVSwitch 1 <-- 9 links -----+

```

对每个 GPU: 18 links x 100 GB/s = 1.8 TB/s bidirectional

节点 aggregate 注入带宽: 8 x 1.8 TB/s = 14.4 TB/s

注意：两个 NVSwitch 都覆盖全部 8 个 GPU，因此不要把它画成“GPU 必须先经过 Switch 0 再经过 Switch 1”的串联网络。

4.2 3.2 与 HGX H100 的物理差异

- HGX H100 Baseboard 使用 4 个 NVSwitch。
- 单 GPU 的 18 条链路在四个 switch 上常见为 4/5/5/4 分布。
- HGX B300 使用 2 个 NVSwitch，变为 9/9 分布。

- 对上层 NCCL 而言，两代节点内都提供全互联交换结构，但底层链路数、switch 数量、带宽和故障域不同。

来源：[NVIDIA Fabric Manager - HGX H100/B200/B300 拓扑](#)。

4.3 3.3 对通信迁移的实际影响

1. 节点内 **collective** 的上限提高，但小消息延迟不会自动减半。Kernel launch、协议、channel 数量、同步和拓扑发现仍会主导小尺寸。
2. **AllReduce 不是唯一需要测的**。MoE 重点测 AllToAll/AllToAllV、AllGather、ReduceScatter；PD 分离与 KV 迁移还要测 P2P 和 NIC 路径。
3. **GPU-NIC affinity 仍然关键**。DGX B300 的 ConnectX-8 直接通过 PCIe Gen6 连接 GPU tray，但 NUMA、IRQ、GDR、rail 绑定错误仍会拖慢跨节点。
4. **14.4 TB/s 不能直接作为应用通信带宽**。NCCL algbw、busbw 与产品 aggregate 口径不同，验收时必须统一定义。
5. **先验证 Fabric Manager 和链路健康，再看模型**。否则模型抖动可能被误判为 kernel 问题。

建议基线命令：

```
nvidia-smi topo -m
nvidia-smi nvlink -s
nvidia-smi -q
sudo dcgmi diag -r 3
```

nccl-tests 示例；按实际构建路径和 MPI 环境调整

```
./build/p2pBandwidthLatencyTest
./build/all_reduce_perf -b 8 -e 8G -f 2 -g 8
./build/all_gather_perf -b 8 -e 8G -f 2 -g 8
./build/reduce_scatter_perf -b 8 -e 8G -f 2 -g 8
./build/alltoall_perf -b 8 -e 8G -f 2 -g 8
```

开源工具：[NCCL Tests](#)、[DCGM](#)。

5 4. 从 WGMMA 到 TCGen05/TMEM

5.1 4.1 为什么不能把 Hopper kernel 原样带过去

Hopper 的高性能 GEMM 主循环通常围绕 TMA + wgmma.mma_async + warpgroup accumulators 构建。Blackwell 数据中心 GPU 的主路径改为：

```
GMEM -- TMA --> SMEM(A/B)
      |
      v
tcgen05.mma / UMMA
      |
      v
      TMEM(D)
      |
      tcgen05.ld
      |
      v
      RMEM epilogue --> GMEM
```

关键差异：

- tcgen05.mma 是异步指令，计算 $D = A * B + D$ 或 $D = A * B$ 。
- A 可来自 TMEM 或 SMEM；B 来自当前 CTA/peer CTA 的 SMEM；D 累加到 TMEM。
- cta_group::1 由当前 CTA 的一个线程发起；cta_group::2 由 CTA pair 中一个线程发起，peer CTA 必须仍处于 active 状态。
- TMEM 是 Tensor Core 专用片上存储，Colfax 教程给出的容量为 256 KB/SM；TMEM 需要显式 allocate/deallocate。

- epilogue 前要将 accumulator 从 TMEM 搬回 register memory；一个 warp 只能访问 TMEM 的部分 lanes，通常需要 warpgroup 协同完成 copy-out。
- 传统 register pressure 模型会变化：MMA accumulator 不再占用常规寄存器，但 TMEM 生命周期、barrier 和 copy-out 成为新约束。

权威资料：[PTX ISA - TensorCore 5th Generation](#)、[CUTLASS Blackwell Functionality](#)、[Colfax TMEM 教程](#)。

5.2 4.2 指令吞吐应如何理解

CUTLASS 文档把不同 tcgen05.mma 类型相对 Hopper WGMMA 的指令吞吐描述为约 2x-4x：

- TF32、F16/BF16、FP8/MXFP8 类路径：约 2x Hopper 对应 Tensor Core 指令吞吐。
- MXFP4/NVFP4 类路径：约 4x Hopper FP8 Tensor Core 指令吞吐。

这不是 kernel 或模型的必然加速比。要达到接近指令上限，必须同时满足：

- shape 足够大且与合法 tile 对齐；
- TMA/TMEM pipeline 饱和；
- block scale 生成/加载没有成为瓶颈；
- epilogue、quantization 和通信占比低；
- cluster/CTA pair 没有造成 occupancy 或尾部浪费；
- 功耗和频率未被系统限制。

5.3 4.3 迁移审计关键词

建议首先对仓库做静态清点：

```
rg -n "wgmma|mma\.sync|tcgen05|cp\.async\.bulk|mbarrier|cluster|tmem|sm_90a|__CUDA_ARCH__|CUTLASS_ARCH_MMA" .
rg -n "int8|fp8|e4m3|e5m2|nvfp4|mxfp4|mxfp8|block[_-]?scale" .
```

将代码分成四类：

1. 纯 CUDA C++/普通 SIMT：通常可先重新编译，再做性能调优。
2. cuBLAS/cuBLASLt/CUTLASS 调用：先升级到明确支持 SM103 的版本并验证选路。
3. Triton/DSL kernel：检查所用版本是否真正生成 B300 路径，而不是通用 fallback。
4. inline PTX/架构硬编码：风险最高，必须建立独立 SM103 实现或调用库路径。

6 5.1-SM 与 2-SM Cooperative Kernel

6.1 5.1 概念

- **1-SM UMMA**：一个 CTA 在一个 SM 上管理 operand、TMEM accumulator 与同步，使用 cta_group::1。
- **2-SM UMMA / pair-UMMA**：cluster 内相邻 CTA 形成 CTA pair，两个 SM 协作完成一个更大的 MMA tile，使用 cta_group::2。
- CTA pair 通常由 cluster 内 CTA ID 的最低位配对，例如 0/1、2/3。
- 2-SM 常与 TMA multicast 配合，让参与 CTA 分摊全局内存加载并在 cluster 中复用数据。

可信教程：[Colfax - GEMM with Thread Block Clusters](#)、[CUTLASS Blackwell examples](#)。

6.2 5.2 NVFP4/MXFP4 常见合法 tile

CUTLASS 当前 Blackwell 文档列出的 FP4 dense MMA 典型 tile：

模式	M	N	K	Layout
1-SM	128	128/192/256	256	TN
2-SM	256	128/192/256	256	TN

这意味着：

- FP4 的 K 对齐和 TN layout 是实际接口约束，不只是“性能建议”。
- 非整齐 shape 可能需要 padding、split-K、grouping、fallback 或其他 kernel family。
- grouped GEMM/MoE 中专家 token 数分布不规则时，2-SM 可能增加尾部浪费。

资料：[CUTLASS Blackwell Functionality - valid tile shapes](#)。

6.3 5.3 何时优先 2-SM

更可能获益：

- 大 M、大 N、规则 K 的 compute-bound GEMM；
- 多个 CTA 会重复读取相同 operand，TMA multicast 可显著降低流量；
- tile 数足够多，CTA pair 后仍能铺满机器；
- epilogue 较轻，主循环占比高。

更可能负收益：

- decode 阶段小 batch/小 M；
- narrow/skinny GEMM；
- MoE 每专家 token 数波动大；
- grouped GEMM 中大量小 problem；
- fused epilogue 很重，TMEM copy-out 后寄存器/访存占比高；
- cluster 约束导致 residency 下降；
- problem tile 数不是 pair/cluster 的友好倍数。

6.4 5.4 推荐的选择机制

不要全局硬编码 1-SM 或 2-SM。建议建立：

```
key = (arch, dtype, layout, M_bucket, N_bucket, K_bucket,
       epilogue, grouped/ragged, batch_or_tokens)
value = best_kernel_family + tile + cluster + stages
```

- bring-up 默认 1-SM；
- 对真实 shape histogram 做离线 sweep；
- 线上只查表，不在热路径重新编译；
- 保留 1-SM fallback；
- 将长尾 shape 单独分桶，不要让平均值掩盖 P99。

可用工具：[CUTLASS Profiler](#)、[CuTile Autotuning Skill](#)。

7 6. Gather4：是什么、能做什么、不能做什么

7.1 6.1 指令语义

`tile::gather4` 是 TMA tile 模式的一个特化：

- 只支持 2D tensor；
- 从源 tensor 的四个不连续行读取数据；
- 将四行组合成一个连续的 2D destination tile；
- 指令输入包含 dimension 0 的四个坐标和 dimension 1 的一个坐标；
- 不支持 interleaved layout；
- 仍受 TMA tile 对齐、边界、tensor map、swizzle 等约束。

权威资料：[PTX ISA - .tile::gather4 / .tile::scatter4](#)。

7.2 6.2 可探索的工作负载（工程推断）

以下不是 NVIDIA 官方承诺，而是值得 microbenchmark 的候选：

- MoE dispatch 中一次打包 4 个 token/expert row；

- paged KV cache/block table 中四个离散 page row 的搬运；
- embedding/top-k 后四行特征 gather；
- ragged batch 中固定四行一组的数据重排；
- 将手写四次 TMA/load 合并为一次 descriptor-driven gather。

7.3 6.3 什么时候不要用

- 每次不是恰好四行；
- 四行的列区间不一致；
- 行很短，tensor map 与 barrier 成本超过搬运收益；
- index 生成或边界处理占主导；
- 数据本来已经连续，普通 TMA tile 更简单；
- 需要任意数量、任意维度的通用 gather。

7.4 6.4 最小验证矩阵

变量	建议值
row bytes	32 B, 64 B, 128 B, 256 B, 1 KB, 4 KB
行间距	连续、固定 stride、随机 page stride
index 局部性	同一 L2 set 邻近、跨 page、随机
对照	4 次普通 TMA、coalesced load、LDGSTS/手写 copy
输出	ns/op、GB/s、TMA active、L2 hit、barrier stall

8 7. NVFP4、MXFP4、MXFP8 与 Block Scaling

8.1 7.1 Block scaling 的计算形式

Blackwell 硬件 block scaling 可抽象为：

$$D = (A * scale_A) @ (B * scale_B) + C$$

核心目标是以更细粒度 scale 缓解窄格式动态范围不足，同时由 Tensor Core 在 MMA 路径中完成缩放。见 [Colfax Block Scaling 教程](#)。

8.2 7.2 格式对比

格式	元素格式	block 粒度	block scale	额外 scale	标准/归属	约有效存储*
FP8	E4M3/E5M2	通常 tensor/recipe 级	FP32 等	依 recipe	NVIDIA/行业通用	8 bit/value + metadata
MXFP8	常用 E4M3	32 consecutive values	E8M0 (2 的 幂)	通常无全局 FP32	OCP MX	8.25 bit/value
MXFP4	E2M1	32 values	E8M0	通常无全局 FP32	OCP MX	4.25 bit/value
NVFP4	E2M1	16 values	E4M3	每 tensor FP32 scale	NVIDIA	约 4.5 bit/value

* 这里只把 element + block scale 摊销到每个 value；忽略 layout padding、alignment、全局 scale、双份 transpose 以及运行时 workspace。

来源：[Colfax Block Scaling 教程](#)、[Transformer Engine FP8/FP4 Primer](#)、[Microscaling Data Formats 论文](#)。

8.3 7.3 MXFP8

特点：

- 每 32 个连续值一个 E8M0 scale；E8M0 仅表达 2 的幂。
- 相比 tensor-wide FP8，对局部 outlier 更稳，通常可以更多使用 E4M3。
- reduction 维度必须满足硬件的 consecutive/layout 约束。
- 转置不是简单 bitwise transpose：训练中往往要从高精度原始 tensor 分别生成 rowwise 和 columnwise/transpose 版本，增加量化和存储成本。

工程建议：

- 将 activation、weight、gradient 分开校准；
- 把 quantization 时间计入 autocast benchmark；
- 检查 M/N/K 的 32 对齐与 scale layout；
- 对小 GEMM 允许 FP8/BF16 fallback。

资料：[Transformer Engine MXFP8](#)。

8.4 7.4 MXFP4

特点：

- E2M1 value + 每 32 个值一个 E8M0 scale；
- OCP 标准格式，跨厂商/软件生态的规范意义更强；
- scale 粒度比 NVFP4 粗，scale 只有 2 的幂；
- 与 NVFP4 不能直接复用同一 packed buffer 或 scale tensor。

适合优先评估的场景：

- 已有 OCP MX 格式数据/模型资产；
- 需要标准化 interchange；
- 模型对 block-32 与 power-of-two scale 误差可接受；
- 软件库对目标 op 已有原生 MXFP4 kernel。

8.5 7.5 NVFP4

特点：

- E2M1 value；每 16 个值一个 E4M3 block scale；另有 per-tensor FP32 scale。
- block 更小、scale 有 mantissa，通常比 MXFP4 有更细的局部表达能力，但 metadata 更多。
- Transformer Engine 的训练 recipe 不仅是 dtype：还包含 stochastic rounding、权重 2D 16x16 scale、Random Hadamard Transform、部分层保留更高精度等策略。
- CUTLASS 当前 FP4 Tensor Core 主路径通常要求 TN layout 和特定 tile。

资料：[Transformer Engine NVFP4](#)、[Pretraining Large Language Models with NVFP4](#)。

8.6 7.6 量化迁移推荐顺序

阶段 A: BF16 correctness + performance baseline

阶段 B: FP8 与 H200 结果/收敛/精度 parity

阶段 C: MXFP8，先 GEMM microbench，再 layer，再 model

阶段 D: NVFP4 inference 或 pre-quantized weights

阶段 E: NVFP4 training recipe / MXFP4，单独做长程质量验证

每一阶段至少保留：

- 与 BF16/FP8 的逐层误差；
- 端到端 task accuracy/perplexity/reward/acceptance；
- NaN/Inf、amax/scale saturation、zero rate；
- quantize/dequantize 开销；
- packed layout 与 checkpoint 兼容性；

- fallback op 列表。

9 8. B300 软件技术栈

9.1 8.1 分层技术栈

层	重点组件	B300 迁移关注点
固件/驱动	GPU firmware、BMC、R610+/认证驱动	与 CUDA/NGC/OEM 矩阵一致；不要只满足最低版本
Fabric	Fabric Manager、NVLink 管理、DCGM	2-NVSwitch 拓扑、link health、partition 状态
OS/容器	DGX OS 7、Ubuntu 24.04、RHEL/Rocky、NGC	锁镜像 digest、驱动兼容、host/container NCCL 一致
CUDA Core	CUDA 13.x、PTX 9.x、NVCC/NVRTC	sm_103a、TMEM/TCGen05、已知正确性修复
Math	cuBLAS/cuBLASLt、cuDNN、cuSPARSE	SM103 algorithm availability、FP4/MXFP8、grouped GEMM
Kernel C++	CUTLASS/CuTe C++	B300 SM103 kernel、1/2-SM、TMEM、block scaling
Kernel Python	CuTe DSL、cuTile/CUDA Tile、Triton	架构选路、AOT/JIT、autotune cache、fallback
Precision	Transformer Engine、ModelOpt	MXFP8/NVFP4 recipe、scale layout、质量回归
Distributed	NCCL、NVSHMEM、UCX/NIXL	NVLink/NVSwitch、IB/Ethernet、GDR、AllToAll
Runtime	TensorRT-LLM、Dynamo、vLLM、SGLang	B300 release matrix、FP4 KV/weights、EP/TP/PP
Kernels	FlashInfer、FlashAttention、DeepGEMM 等	是否有 sm_103 实测路径，不只“能编译”
Profiling	Nsight Compute、Nsight Systems、DCGM	TMEM/TMA/cluster、通信 overlap、功耗/降频

9.2 8.2 建议的版本策略

- B300 最低 CUDA 支持是 13.0；工程上不要以“最低可运行”作为生产版本。
- 本报告时点的 CUDA Release Notes 为 13.3 Update 1；使用供应商认证的当前 CUDA 13.x NGC 容器并固定 digest。
- CUTLASS 使用明确列出 SM103/B300 支持的当前稳定 release/branch；SM100-only 示例只能作为原理参考。
- Transformer Engine 使用明确列出 B300、MXFP8、NVFP4 的版本。
- TensorRT-LLM 逐条核对 release notes 中 B300/GB300、量化、PP/EP、disaggregated serving 的 known issues。
- PyTorch/Triton/vLLM/SGLang/FlashInfer 不要凭“Blackwell supported”推断 B300 所有 kernel 均已优化；对关键 op 查看实际 cubin/SASS 与 benchmark。

9.3 8.3 CUDA 13.x 已知正确性风险

必须进入迁移测试计划的公开问题：

1. CUDA 13.2 Update 1 的 cuBLAS 曾可能忽略 NVFP4 tensor-wide scaling，造成错误结果；需安装对应 patch 或使用后续已修复版本。
2. 后续 release 修复了 B300/GB300 上 NVFP4 M 非 64 倍数时的错误结果。
3. 曾存在 cuBLASLt 与其他 TMEM kernel 并发时的错误/数据损坏问题，版本与 algorithm ID 相关。
4. CUDA 13.3 仍有部分 cuBLASLt heuristic/capture、GEMV-like shape 等 known issues；升级后仍需阅读当期 release notes，而不是只看“resolved”。

建议专项回归：

$M \% 64 = \{0, 1, 2, 31, 32, 63\}$

N/K 覆盖最小、典型、超大 leading dimension

beta = {0, 1, 非零随机}

tensor-wide scale / block scale / mixed scale

单 stream / 多 stream / CUDA Graph capture

与自定义 TMEM kernel 并发

grouped GEMM: 含 K=0、小 group、极不均匀 group

资料: [CUDA 13.3 Update 1 Release Notes](#)。

10 9. 推荐文档、教程、论文与开源项目

10.1 9.1 第一优先级：官方架构与 ISA

1. [Inside NVIDIA Blackwell Ultra](#) - 芯片、HBM、NVLink、峰值、Attention SFU。
2. [DGX B300 User Guide](#) - 节点硬件、功耗、网络、OS。
3. [NVIDIA Fabric Manager User Guide](#) - H100/B200/B300 NVLink 物理拓扑。
4. [PTX ISA 9.x](#) - tcgen05、TMEM、TMA gather4、target ISA。
5. [CUDA Programming Guide](#) - cluster、TMA、CUDA 基础模型。
6. [CUDA Toolkit Release Notes](#) - B300/cuBLAS 正确性和性能修复。

10.2 9.2 第二优先级：CUTLASS/CuTe 与可信教程

1. [CUTLASS Overview](#) - B300 CC10.3、最低 CUDA、DSL/C++ 总览。
2. [CUTLASS Blackwell Functionality](#) - TCGen05、1/2-SM、tile、数据类型。
3. [CUTLASS Changelog](#) - SM103 支持与 bug fix。
4. [NVIDIA CUTLASS GitHub](#) - 代码与 examples。
5. [SM103 FP4 Ultra GEMM](#) - B300 dense FP4 示例。
6. [SM103 FP4 Ultra Grouped GEMM](#) - B300 grouped FP4 示例。
7. [CuTe Blackwell tutorial examples](#) - TMEM/TMA/1-SM/2-SM。
8. [2-SM CuTe example](#) - CTA pair 实现参考；需再验证/适配 SM103。
9. [Colfax Part 1: Tensor Memory](#) - 从 Hopper kernel 视角理解 UMMA/TMEM。
10. [Colfax Part 2: Thread Block Clusters](#) - TMA multicast、1-SM/2-SM、同步。
11. [Colfax Part 4: Block Scaling](#) - NVFP4/MXFP4/MXFP8 与 scale layout。

10.3 9.3 低精度与训练/推理 recipe

1. [Transformer Engine FP8/FP4 Primer](#)
2. [Transformer Engine MXFP8](#)
3. [Transformer Engine NVFP4](#)
4. [Transformer Engine GEMM Speedups](#)
5. [NVIDIA TransformerEngine GitHub](#)
6. [NVIDIA Model Optimizer](#)
7. [Microscaling Data Formats for Deep Learning](#)
8. [Pretraining Large Language Models with NVFP4](#)
9. [Microsoft microscaling](#) - OCP MX 参考实现/实验代码。

10.4 9.4 Tile/DSL 与迁移生产力

1. [cuTile Python Quickstart](#)
2. [cuTile Python GitHub](#)
3. [NVIDIA TileGym](#) - kernel 教程、benchmark、LLM 集成。
4. [CUDA 13.3 Tile Programming Blog](#)
5. [CuTe DSL performance/productivity blog](#)
6. [OpenAI Triton on Blackwell](#)

10.5 9.5 Runtime、通信与可观测性

- 1. [TensorRT-LLM Release Notes](#)
- 2. [TensorRT-LLM GitHub](#)
- 3. [NCCL Tests](#)
- 4. [NVIDIA DCGM](#)
- 5. [Nsight Compute](#)
- 6. [Nsight Systems](#)
- 7. [vLLM](#)
- 8. [SGLang](#)
- 9. [FlashInfer](#)
- 10. [FlashAttention](#)

11 10. NVIDIA Skills：推荐安装与使用方式

11.1 10.1 Skills 的定位

NVIDIA Verified Agent Skills 是给兼容 AI coding agent 使用的可移植操作规程、参考资料和脚本。它能帮助代理按 NVIDIA 推荐方法完成 kernel、训练、推理和排障任务，但不能替代 PTX/CUDA/CUTLASS 文档、单元测试或 benchmark。

官方入口：[NVIDIA Skills Documentation](#)、[skills.sh catalog](#)、[NVIDIA/skills GitHub](#)。

安装示例：

```
npx skills add nvidia/skills
npx skills add nvidia/skills --agent codex
```

11.2 10.2 对 H->B 迁移最相关的 Skills

优先级	Skill	用途
P0	tilegym-cutile-python	cuTile 语法、执行模型与常见 kernel 模式
P0	tilegym-adding-cutile-kernel	新增 cuTile kernel 的标准流程
P0	tilegym-improve-cutile-kernel-perf	性能诊断与优化
P0	tilegym-cutile-autotuning	shape/arch 感知的 tune-once/cache/launch
P1	dynamo-interconnect-check	GPU/NIC/互联 bring-up 检查
P1	dynamo-troubleshoot	推理服务排障
P1	dynamo-recipe-runner	标准化运行 inference recipe
P1	nemo-mbridge-perf-cuda-graphs	训练 CUDA Graph 性能路径
P1	nemo-mbridge-perf-memory-tuning	显存/activation/allocator 调优
P1	nemo-mbridge-perf-parallelism-strategies	TP/PP/DP/CP 选择
P1	nemo-mbridge-perf-tp-dp-comm-overlap	通信计算重叠
P1	nemo-mbridge-perf-moe-comm-overlap	MoE dispatch/communication overlap
P1	nemo-mbridge-perf-moe-hardware-configs	MoE 与硬件配置匹配
P1	nemo-mbridge-perf-moe-dispatcher-selection	dispatcher 选型

训练类 skill 以 [NVIDIA skills catalog](#) 中当前名称为准，因为 catalog 会更新。

11.3 10.3 使用规则

- 把 Skills 当作“可重复的工程 checklist”，不要把其默认参数当成 B300 最优参数。
- agent 生成的 kernel 必须通过 reference、随机 shape、边界 shape、compute-sanitizer 与性能基线。
- autotune 结果的 cache key 必须包含 architecture、dtype、shape bucket、layout、epilogue 和关键 runtime 参数。
- 迁移期间固定 Skill commit/tag；否则操作规程本身变化会污染复现性。

12 11. 公开性能数据

12.1 11.1 B300 理论/产品规格

指标	公布值	口径
单 GPU dense NVFP4	15 PFLOPS max	芯片最大值
单 GPU sparse NVFP4	20 PFLOPS max	芯片最大值；不是 dense 的 2x
单 GPU dense FP8	5 PFLOPS max	芯片最大值
DGX/HGX B300 dense FP4	108 PFLOPS	8-GPU 系统产品规格
DGX/HGX B300 sparse FP4	144 PFLOPS	8-GPU 系统产品规格
DGX/HGX B300 sparse FP8	72 PFLOPS	dense 为 36 PFLOPS
HBM	up to 8 TB/s/GPU	芯片上限
NVLink	1.8 TB/s/GPU bidirectional	产品互联规格

来源：Blackwell Ultra blog、DGX B300。

12.2 11.2 Transformer Engine GEMM 公测数据

Transformer Engine 在 B300 上给出的一个 5B 模型配置（hidden=4096、intermediate=16384、24 layers）示例：

模式	相对 BF16	是否包含每步量化开销
MXFP8 autocast	约 1.44x	包含
FP8 DelayedScaling autocast	约 1.61x	包含
NVFP4 autocast	约 2.03x	包含
NVFP4 pre-quantized	约 3.55x	不包含

官方文档同时明确强调：低精度加速高度依赖 shape，大 GEMM 能摊薄量化成本，小 GEMM 可能收益很小甚至变慢。来源：TE GEMM Speedups。

12.3 11.3 CUDA 软件版本带来的同硬件增益

CUDA 13.3 release notes 公布：

- Blackwell Ultra FP4 matmul 在一组问题上 geomean 约 +5%，部分小问题最高约 +7%。
- Blackwell/Blackwell Ultra TF32 matmul geomean 约 +27%，部分小问题最高 3.5x。
- TMA-based SYMV 在 Hopper/Blackwell/Blackwell Ultra 上最高约 +27% geomean。

这说明迁移 benchmark 必须锁定软件栈；“H 卡旧栈 vs B 卡新栈”无法把硬件收益与软件收益分开。来源：CUDA Release Notes。

12.4 11.4 MLPerf Inference 6.0：GB300 参考

Lambda 公布的 MLCommons verified closed-division 结果：

- 4-GPU GB300 compute tray，GPT-OSS-120B：Offline 60,220 tok/s，Server 53,463 tok/s。

- 相对 4 x B200 baseline 的 1.29x/1.22x 是通过将一个 8-GPU B200 结果减半得到的估算对比，因为没有相应 4-GPU B200 submission。
- 同一 B200 硬件仅通过 TensorRT-LLM/CUDA 软件升级，在 Llama 3.1 8B 上出现最高约 9% 提升。

因此该数据能说明 Blackwell Ultra 与软件成熟度的方向，但不能作为 HGX B300 对 H200 的直接验收值。来源：[Lambda MLPerf Inference v6.0](#)、[MLCommons Inference](#)。

12.5 11.5 MLPerf Training 6.0: HGX B300 实测

Nebius 公布并引用 MLCommons verified closed-division entries:

Workload	8 x HGX B300	64 x HGX B300
Llama 3.1 8B pretraining	72.01 min	14.42 min
GPT-OSS 20B MoE pretraining	83.62 min	23.48 min
FLUX.1	-	46.71 min

该数据适合判断 B300 的可运行性和系统调优成熟度，但不同规模下可能使用不同全局 batch、并行方案和收敛过程，不能简单按时间反推通信 scaling efficiency。来源：[Nebius MLPerf Training v6.0](#)、[MLCommons Training](#)。

12.6 11.6 B200 vs H200 的公开下限参考

B300 的直接 H200 等条件公开对比仍有限。可把 B200 的历史数据作为 “Blackwell 软件路径已能带来显著收益” 的参考，而不是 B300 结论：

- NVIDIA MLPerf Inference v5.0 中，8 x B200 相对 8 x H200 的部分 LLM 场景约 2.1x-3x，SDXL 约 1.6x；精度、实现和工作负载不同，不能外推所有模型。
- NVIDIA 的 Triton Blackwell 资料显示 FP16 FlashAttention 在 B200 对 Hopper 的特定实现中最高约 1.5x。

来源：[NVIDIA MLPerf v5.0 blog](#)、[OpenAI Triton on Blackwell](#)。

12.7 11.7 公开数据使用规则

- 不比较不同精度的 PFLOPS。
- 不把 sparse 峰值当 dense。
- 不把 MLPerf Offline 与 Server 直接相除比较。
- 不把 rack-scale 结果无说明地除以 GPU 数。
- 不把 pre-quantized GEMM 当包含量化的端到端收益。
- 必须记录 ISL/OSL、batch/concurrency、KV precision、weight precision、TP/PP/EP、CUDA/TRT-LLM/TE 版本、功耗。

13 12. 迁移执行计划

13.1 Phase 0: 冻结 Hopper 基线

需要收集：

- GPU SKU：H100/H200、SXM/PCIe、显存、功耗上限、时钟。
- 驱动、CUDA、cuBLAS、NCCL、PyTorch、Triton、框架 commit、容器 digest。
- 所有自定义 kernel 与 inline PTX 清单。
- 真实 shape histogram：GEMM 的 M/N/K、group size、batch、sequence、prefill/decode。
- 通信 histogram：collective 类型、message bytes、group size、频率。
- 质量指标：loss/perplexity、下游 accuracy、RL reward/KL、MTP acceptance、生成一致性。
- 性能指标：step time、tok/s、TTFT、TPOT/ITL、P50/P95/P99、power/token。

13.2 Phase 1: B300 平台 bring-up

1. 固定 OEM 认证驱动、Fabric Manager、CUDA/NGC 容器。
2. 确认实际 compute capability、SM 数、HBM 可用量、TGP 和 clocks。
3. 跑 DCGM 诊断、NVLink link state、拓扑。
4. 跑 P2P 与 NCCL collectives，保存原始 CSV。
5. 验证 GPUDirect RDMA、NIC rail、NUMA/CPU affinity。
6. 进行 1-4 小时 burn-in，检查 Xid/ECC/thermal/power throttle。

13.3 Phase 2: 正确性优先的 kernel 迁移

1. BF16/FP32 reference 先通。
2. 用 cuBLASLt/CUTLASS SM103 kernel 替代最危险的 Hopper inline PTX。
3. 默认 1-SM，随机 shape + 边界 shape + adversarial scale 测试。
4. 加入 TMEM access check、compute-sanitizer、racecheck（可用处）。
5. 测 CUDA Graph、multi-stream、并发自定义 kernel。
6. 对每个 fallback 记录“原因 + 性能 + 正确性”。

13.4 Phase 3: 精度与 1-SM/2-SM 调优

1. FP8 parity。
2. MXFP8 autocast 和 pre-quantized 分开测。
3. NVFP4 先推理/weight-only，再扩展 activation/training。
4. 按 shape bucket autotune 1-SM/2-SM、tile、cluster、stage。
5. 对 MoE grouped GEMM 单独按 expert token histogram 调优。
6. gather4 仅在真实四行 gather pattern 上 microbenchmark。

13.5 Phase 4: 模型与分布式验证

训练：

- step time、tokens/s/GPU、MFU；
- TP/DP/PP/CP/EP overlap；
- optimizer、activation checkpoint、checkpoint IO；
- loss curve、收敛、梯度统计和长程稳定性。

推理：

- prefill/decode 分开；
- ISL x OSL x concurrency 矩阵；
- TTFT、TPOT/ITL、tok/s、req/s、P99；
- KV cache capacity、fragmentation、reuse/offload；
- speculative decoding/MTP acceptance；
- MoE load balance、dispatch/combine、AllToAll 长尾。

13.6 Phase 5: 生产可靠性

- 24 小时以上 soak test；
- 驱动/CUDA/容器/模型/量化 recipe 全部锁版本；
- 监控 ECC、Xid、NVLink、HBM temperature、power cap、clocks；
- 故障注入：单 rank 失败、网络抖动、OOM、worker restart；
- 建立 H/B 双平台回归与性能 guardrail。

14 13. Benchmark 设计

14.1 13.1 Microbenchmark 层

类别	必测内容	工具
Memory	HBM copy、L2、不同 stride/alignment	自定义 CUDA、Nsight Compute
GEMM	BF16/FP8/MXFP8/MXFP4/NVFP4； 真实 M/N/K	CUTLASS Profiler、TE benchmark
1/2-SM	同一 shape、同一精度、同一 epilogue A/B	CUTLASS/CuTe
Quantization	quantize、pack、transpose、scale generation	TE/自定义 benchmark
Gather4 Attention	gather4 vs 4x TMA vs manual load prefill/decode、MHA/GQA/MLA、varlen	PTX/CuTe microbench FlashInfer/FA/TRT-LLM
Comm	P2P、AR/AG/RS/A2A，真实 bytes	nccl-tests

14.2 13.2 End-to-end 层

推理至少覆盖：

维度	建议值
ISL	128, 1K, 4K, 16K, 64K（按模型限制）
OSL	1, 32, 256, 1K
concurrency	1, 4, 16, 64, saturation
场景	offline throughput、online latency、interactive
精度	BF16、FP8、MXFP8/NVFP4 可用组合
并行	TP1/2/4/8，MoE EP 组合，必要时 PP
输出	TTFT、TPOT/ITL、tok/s、req/s、P99、J/token

训练至少覆盖：

- 单 GPU kernel upper bound；
- 8 GPU 单节点；
- 2/4/8 节点 scaling；
- dense 与 MoE 各一个代表模型；
- short context 与 long context；
- FP8 baseline 与 MXFP8/NVFP4 recipe；
- 稳定性运行而非只跑 20 step warmup。

14.3 13.3 Nsight 观察项

不要只看“Tensor Core utilization”一个数字。至少联动观察：

- achieved FLOP/s 与**对应精度/稀疏口径**的峰值；
- SM active、eligible warps、occupancy；
- HBM throughput、L2 hit、sector/request；
- TMA activity 与 producer/consumer overlap；
- TMEM 生命周期、copy-out 与 barrier stall；
- cluster/CTA pair 的 residency 与尾块；
- epilogue、quantization、layout transform 占比；
- NCCL kernel 与 compute overlap；
- GPU clocks、power、thermal throttle；
- CPU launch、Python/runtime gap、CUDA Graph capture 覆盖率。

15 14. 重点风险清单

风险	严重度	检测方式	缓解
把 B300 当 SM100	Critical	cubin/arch/feature audit	独立 SM103 build 与 CI
Hopper WGMMMA inline PTX	Critical	rg wgmma + 编译/运行	CUTLASS/SM103 重写或 library fallback
INT8 TCGen05 假设	Critical	PTX target notes + SASS	检查实际 kernel family, 不默认原生
NVFP4 已知 cuBLAS bug	Critical	版本审计 + M%64 sweep	patched CUDA/ cuBLAS, 锁镜像
NVFP4/MXFP4 buffer 混用	Critical	metadata/layout validator	重新量化/打包, 格式带 schema/version
2-SM 全局强制	High	P50/P99 shape benchmark	shape-aware autotune + 1-SM fallback
只测大方阵 GEMM	High	对比真实 histogram	把 decode/MoE 小 shape 纳入权重
aggregate NVLink 误解	High	NCCL algbw/busbw	明确指标定义、实测 collectives
只测单节点	High	2-8 node scaling	NIC/GDR/rail/NUMA 专项
只看 tok/s 不看质量	High	task/loss/acceptance	质量门禁先于性能门禁
JIT 首次编译污染	Medium	cold/warm 分开	AOT 或缓存, 记录首次延迟
软件版本漂移	High	digest/commit manifest	可复现环境清单
功耗/散热降频	High	DCGM/power/clocks	PDU/冷却预验收、功耗归一化

16 15. 第一阶段十个工作日建议

16.1 Day 1-2: 资产与平台

- 冻结 H200 基线; 导出环境和 shape/communication histogram。
- B300 跑 DCGM、topology、NVLink、P2P、NCCL。
- 记录实机 SM、HBM、TGP、clocks、BIOS/BMC/Fabric Manager。

16.2 Day 3-4: 编译与正确性

- 生成 H/B fatbin 或两套 binary; 确认 sm_103a code object。
- 审计 WGMMMA/inline PTX/INT8/FP8 自定义路径。
- BF16 reference + boundary shape + multi-stream/TMEM 并发测试。

16.3 Day 5-6: GEMM 与精度

- CUTLASS/TE 跑真实 shape sweep。
- 比较 1-SM/2-SM; 建立首版 lookup table。
- FP8 parity; 测 MXFP8 与 NVFP4 的 quantization-inclusive 和 pre-quantized 两套结果。

16.4 Day 7-8: 模型

- 推理: prefill/decode、ISL/OSL/concurrency、TTFT/TPOT/P99。
- 训练: 单节点 step time、MFU、显存、loss parity。
- MoE: expert histogram、grouped GEMM、AllToAll、load imbalance。

16.5 Day 9: 多节点与可靠性

- 2/4/8 节点 collective + 模型 scaling。
- 检查 NIC affinity、GDR、rail、CPU pinning。
- 启动 12-24 小时 soak。

16.6 Day 10: Go/No-Go

形成一张验收表：

门槛	建议
Correctness	全部 reference/边界/并发测试通过
Quality	与 H200 baseline 的差异在业务预算内
Single-node perf	按真实 workload 加权后达到目标
Multi-node scaling	无明显链路/rail/尾延迟异常
Memory	峰值显存、fragmentation、KV/activation 达标
Energy	tok/J 或 samples/J 达标，不只看绝对吞吐
Stability	soak 无 Xid/ECC/NVLink/数据损坏
Reproducibility	container digest、commit、config、seed 完整

17 16. 最终建议

对你们这次 H 卡到 B300 的迁移，建议把工作拆成三条并行主线：

1. **平台线**：驱动/Fabric Manager/NCCL/NIC/DCGM，目标是证明机器和网络健康。
2. **Kernel 线**：sm_90a 与 sm_103a 分离，先 CUTLASS/库 + 1-SM，再逐 shape 引入 2-SM、TMEM、block scaling、gather4。
3. **模型/数值线**：BF16/FP8 parity 后逐步上 MXFP8/NVFP4，以质量和长尾延迟作为硬门禁。

最容易踩的坑不是“B300 理论算力没吃满”，而是：

- 旧 Hopper 架构特化代码在 B300 上 silently fallback；
- 低精度 layout/scale 语义混淆；
- CUDA/cuBLAS 已知问题导致错误结果；
- 2-SM 对平均大 shape 加速，但拖慢大量 decode/MoE 长尾；
- 只看单 kernel 或 Offline tok/s，忽略通信、量化、runtime gap、P99 与功耗。

因此，第一版目标不是榨干 15 PF NVFP4，而是建立一套可复现的 SM103 正确性基线、真实 shape benchmark 与架构感知 autotune 体系。这套体系建立后，B300 的 HBM、NVLink、TMEM/TCGen05、MXFP8/NVFP4 才能稳定转化成模型收益。

18 附录 A：环境清单模板

```
hardware:
  source_gpu: H200_SXM
  target_gpu: B300_SXM
  gpu_count: 8
  target_compute_capability: "10.3"
  target_sm_count: "from nvidia-smi/deviceQuery"
  hbm_visible_gib: "from runtime"
  power_limit_w: "from nvidia-smi -q"

software:
  driver: ""
  firmware: ""
  fabric_manager: ""
```

```
cuda: ""
cublas: ""
nccl: ""
pytorch: ""
triton: ""
cutlass_commit: ""
transformer_engine: ""
runtime: "TensorRT-LLM/vLLM/SGLang"
container_digest: ""

benchmark:
  git_commit: ""
  config_hash: ""
  seed: ""
  precision: ""
  model: ""
  parallelism: ""
  isl_osl_concurrency: ""
  warmup_and_duration: ""
```

19 附录 B：资料入口总表

19.1 Hardware / system

- [Blackwell Ultra architecture deep dive](#)
- [DGX B300 User Guide](#)
- [DGX B300 product specifications](#)
- [HGX platform specifications](#)
- [Fabric Manager and NVLink topology](#)

19.2 CUDA / ISA / kernels

- [PTX ISA](#)
- [CUDA Programming Guide](#)
- [CUDA Release Notes](#)
- [CUTLASS documentation](#)
- [CUTLASS Blackwell functionality](#)
- [CUTLASS changelog](#)
- [CUTLASS GitHub](#)
- [CUTLASS Profiler](#)
- [SM103 FP4 GEMM](#)
- [SM103 FP4 Grouped GEMM](#)
- [CuTe Blackwell examples](#)

19.3 Colfax tutorials

- [Tensor Memory and UMMA](#)
- [Thread Block Clusters and 2-SM UMMA](#)
- [Hardware-supported Block Scaling](#)
- [Hopper TMA background](#)

19.4 Precision / papers

- [Transformer Engine](#)
- [Transformer Engine FP8/FP4 primer](#)
- [MXFP8](#)

- [NVFP4](#)
- [GEMM precision speedups](#)
- [Microscaling Data Formats for Deep Learning](#)
- [Pretraining Large Language Models with NVFP4](#)
- [Microsoft microscaling](#)

19.5 CUDA Tile / DSL / Skills

- [cuTile Python Quickstart](#)
- [cuTile Python GitHub](#)
- [TileGym](#)
- [NVIDIA Skills docs](#)
- [NVIDIA Skills catalog](#)
- [NVIDIA Skills GitHub](#)
- [CuTile Autotuning Skill source](#)

19.6 Runtime / performance / tools

- [TensorRT-LLM Release Notes](#)
- [TensorRT-LLM GitHub](#)
- [NCCL Tests](#)
- [DCGM](#)
- [Nsight Compute](#)
- [Nsight Systems](#)
- [MLCommons Inference](#)
- [MLCommons Training](#)
- [Lambda MLPerf Inference v6.0](#)
- [Nebius MLPerf Training v6.0](#)
- [NVIDIA MLPerf Inference v5.0](#)